

E-Commerce Full Stack Assignment

Overview

This document describes the requirements for building a mini full stack e-commerce application. The assignment evaluates both frontend and backend skills, including REST API design, data modeling, server-side pagination/search/filtering, SSR/SSG, URL-based state management, infinite scrolling, layout implementation, and global state handling.

Objective

Build a full stack e-commerce application consisting of a backend REST API (with a database) and a frontend with the following pages: - Product List Page - Product Detail Page - Cart Page

The focus is on correctness, structure, and real-world full stack practices.

Recommended Tech Stack

- **Framework:** Next.js (App Router or Pages Router)
 - **Language:** TypeScript/Javascript
 - **Styling:** CSS / CSS Modules / SCSS / Tailwind CSS
 - **State Management:** Context API / Zustand / Redux Toolkit
 - **Data Fetching:** Fetch API or Axios
 - **Backend:** Any REST stack (.NET / ASP.NET Core, Python / FastAPI, Node, etc.)
 - **Database:** PostgreSQL or SQL Server
-

Data Source & Backend (Mandatory)

All product data consumed by the frontend must come from your own backend API — not from a public mock API such as DummyJSON. You may seed your database from a public dataset (for example, DummyJSON) as a one-time import, but every runtime request must hit your API. See **Backend Requirements** below for the API contract and data model.

Backend Requirements

The backend is a graded part of this assignment. The technology choice (language, framework, and database) is entirely up to you, as long as the requirements below are met.

General

- Expose a REST API in a stack of your choice (.NET / ASP.NET Core, Python / FastAPI / Django, Node / Express / NestJS, etc.).
- Persist data in a relational database (PostgreSQL, SQL Server, MySQL, etc.). No hardcoded JSON files or in-memory arrays at runtime.
- Provide a seed/migration script that creates the schema and loads a catalog of 50+ products across multiple categories.
- Enable CORS for the frontend origin during development.

Data Model (minimum)

- **Product:** id, title, description, price, category, thumbnail/images, stock/rating (optional).
- **Category:** slug, name (a separate table or a derived list, your choice).

Endpoints

- GET /api/products?page={n}&limit={n}&search={q}&category={slug} — paginated list; returns items plus total count / hasMore for infinite scroll.
- GET /api/products/{id} — single product; 404 when not found.
- GET /api/categories — list of categories for the filter.

Behavior

- Pagination, search, and category filtering must run in the database query — never fetch everything and filter on the server in memory or on the client.
- Return proper HTTP status codes and a consistent error response shape. Validate query parameters.
- You may adjust the exact contract (field names, response envelope), but the frontend must consume your own API.

Pages & Features

1. Product List Page (SSR)

Route: /products

Requirements: - Must be server-side rendered (SSR) - Display products in a grid layout - Layout reference: apex4u.com/category/men - Implement infinite scrolling - Sticky filter section on the left

Filters: - Search by product name - Category filter (from API) - Price range (optional)

URL-based State: - Filters must be reflected in the URL - URL should be shareable and restorable

Example: `/products?search=shoe&category=mens-shoes&page=2`

2. Product Detail Page (SSR or SSG)

Route: `/products/[id]`

Rendering: - SSR or SSG (SSG preferred)

Layout: - Left: product images - Right: product details (name, price, description, category) - Add to Cart button

Reference: <https://apex4u.com/product/sprint-mens-sports-shoe-9451aa56>

Cart Interaction: - Add product to cart - Update cart badge count globally

3. Cart Page (Client Side)

Route: `/cart`

Layout Reference: <https://apex4u.com/cart>

Features: - List cart items - Increase/decrease quantity - Remove items - Display subtotal and total

Cart data must persist across page navigation.

State Management

- Use a global state management solution
 - Cart state must be accessible from:
 - Product Detail Page
 - Header Cart Icon
 - Cart Page
-

Key Focus Areas

1. Shareable URL with filter state
2. Infinite scrolling

3. Product grid layout
 4. SSR / SSG implementation
 5. REST API design and clean separation of concerns
 6. Data modeling and database-level pagination, search, and filtering
-

Bonus (Optional)

- Loading skeletons
 - Error and empty states
 - Responsive design
 - SEO metadata
 - Clean folder structure
 - Request validation and consistent error handling on the API
 - Authentication (e.g. JWT) for protected routes
 - API documentation (Swagger / OpenAPI)
 - Automated tests (unit / integration) on the backend
 - Dockerized setup (docker-compose for API + database)
-

Evaluation Criteria

- Code quality and readability
 - Correct backend design and proper API integration
 - Database design and server-side pagination/search/filtering
 - End-to-end integration between frontend and backend
 - Proper SSR/SSG usage
 - URL-driven state handling
 - State management approach
 - Overall user experience
-

Submission Guidelines

- Push code to a public GitHub repository
 - Include a README with:
 - Setup instructions for both frontend and backend (including database setup, migrations, seeding, and environment variables)
 - Architecture explanation
 - Assumptions or limitations
-

Deadline

EOD of 01/07/2026

Focus on clean implementation and best practices rather than pixel perfection. Good luck and happy coding!